Capítulo 12

Confiabilidad y control de flujo

Al terminar este capítulo, entenderás:

- Qué es la confiabilidad y cómo se logra.
- Qué es el control de flujo y por qué es necesario para lograr confiabilidad.
- Qué son y cómo funcionan los protocolos ARQ.
- En qué consiste y cómo funciona la ventana deslizante.
- Qué es la confirmación acumulativa y negativa.

La red no garantiza por sí misma que los datos lleguen correctamente a su destino. Tampoco impide que un emisor envíe datos más rápido de lo que el receptor es capaz de procesarlos. Para resolver cada uno de estos problemas existen dos mecanismos: la confiabilidad y el control de flujo, respectivamente. Ambos se pueden implementar en protocolos de capas diferentes y con enfoques dispares, aunque lo habitual es encontrarlos en la capa de transporte (como TCP) o en la de enlace (como HDLC).

Se dice que una comunicación es fiable o **confiable** si los datos llegan a su destino exactamente tal cual partieron del origen, sin ningún cambio ni omisión, pero también sin partes duplicadas o desordenadas. Hay muchas razones por las que podrían darse todos esos problemas: interferencias, atenuación, retrasos, disparidad de rutas, limitación en el almacenamiento intermedio, etc.

Por otro lado, el **control de flujo** es el mecanismo que permite a los participantes limitar (incluso interrumpir temporalmente) el flujo de datos entre ellos. Aunque el concepto se puede aplicar a varias situaciones, aquí emplearemos la expresión «control de flujo» para un caso específico: el ajuste que el receptor solicita al emisor para evitar que lo sature, es decir, que sobrepase su capacidad de procesar los datos entrantes (error de *overrun*).

La confiabilidad y el control de flujo se suelen estudiar a la vez porque habitualmente se diseñan e implementan de forma conjunta. Además, se

complementan; sin control de flujo, probablemente se producirían pérdidas que el mecanismo de confiabilidad tendría que resolver mediante retransmisiones, desperdiciando capacidad del canal.

Todo mensaje que viaja entre dos dispositivos lo hace a través de algún medio físico, sea el aire, un cable de cobre o una fibra óptica, y por eso está restringido por las limitaciones físicas. Incluso en el vacío, la señal sufre cambios causados por interferencias o ruido electromagnético, y la intensidad de la señal se degrada por la distancia o la resistencia del medio. Por eso, cualquier mensaje que se envíe a través de una red podría llegar modificado o incompleto, o incluso no llegar. Llamamos a todo esto «errores de transmisión» y para solucionarlos existen distintas técnicas, que se clasifican en dos categorías: detección y corrección de errores.

La detección de errores más simple consiste en añadir información redundante al mensaje original, normalmente solo unos pocos bytes. Puede ser algo tan sencillo como un bit de paridad, pero típicamente se utiliza una comprobación de redundancia cíclica (CRC). El emisor aplica ese algoritmo a los datos de partida e incluye el resultado en el propio mensaje. Al llegar, el receptor realiza el mismo cálculo y compara el resultado con el que ha recibido. Si estos valores coinciden, el mensaje es correcto; si difieren, el mensaje ha sufrido algún cambio.

Este tipo de algoritmo solo detecta la alteración, pero no determina en qué parte del mensaje y menos aún cuál fue la alteración. Este es el caso del FCS de Ethernet, o del checksum utilizado en las cabeceras de IP, UDP, TCP, etc., que es un valor de tan solo 16 bits. Se adjunta una explicación detallada de este algoritmo en /inet-checksum/checksum.ipynb.

Si la probabilidad de error es baja, lo más sencillo y eficiente es detectar los errores y retransmitir los mensajes afectados. Si por el contrario la probabilidad es alta, o no es posible o conveniente realizar la retransmisión, entonces se utilizan algoritmos de corrección de errores. Estos algoritmos añaden una cantidad significativa de información redundante a los mensajes, lo que también conlleva recursos de cómputo adicionales. El más conocido de estos algoritmos de corrección es Reed-Solomon, que se utiliza en comunicaciones vía satélite.

Para la retransmisión de mensajes erróneos se utilizan variantes de los protocolos ARQ (Automatic Repeat reQuest) en los que el receptor envía mensajes de **confirmación** (también llamados «reconocimientos» o ACK). Estos protocolos permiten al emisor obtener información actualizada sobre el estado del receptor. Cada vez que el emisor recibe una confirmación tiene constancia de que el receptor está listo para continuar, y envía un nuevo

mensaje, es decir, al mismo tiempo proporciona confirmación y controla el flujo.

Aunque se denominan «protocolos», los ARQ no definen formato o sintaxis concreta para los mensajes. Únicamente definen una serie de reglas y pautas que deben cumplir los mensajes de datos y confirmaciones. Puedes entenderlos como una especie de patrones para crear protocolos confiables concretos.

Las siguientes secciones explican cómo funciona cada uno de los ARQ clásicos.

12.1. Parada y espera

En el protocolo *parada y espera (stop and wait)* el emisor envía un mensaje y espera su confirmación. Si no la recibe en un tiempo determinado, reenvía el mensaje. Si recibe la confirmación, envía el siguiente mensaje.

En detalle:

- El emisor envía un mensaje de datos que incluye un checksum.
- El emisor inicia un temporizador de retransmisión.
- El receptor recibe el mensaje y calcula el checksum.
  - Si coincide, envía una confirmación.
  - Si no coincide, descarta el mensaje.
- Si el emisor recibe la confirmación, cancela el temporizador y envía el siguiente mensaje.
- Si el temporizador expira antes de recibir la confirmación, reenvía el mensaje.

El temporizador debe ajustarse en relación al tamaño de los mensajes y al RTT, dado que un valor demasiado corto provocaría retransmisiones prematuras e innecesarias[1] y, como resultado, el receptor recibe un duplicado del mensaje. Para que el receptor pueda detectar esta situación utiliza un número de secuencia de un único bit: alterna entre 0 y 1. Si el receptor recibe dos mensajes consecutivos con el mismo número de secuencia, sabrá que es un duplicado y lo descartará, pero debe confirmarlo igualmente. El duplicado también aparece cuando se pierde la confirmación y, si el receptor no la repite, el emisor seguirá retransmitiendo ese mensaje indefinidamente.

Hay un detalle que puede resultar confuso y merece la pena aclarar. Es frecuente que muchos protocolos tomen el convenio de enviar en la confirmación el número de secuencia del **siguiente** mensaje esperado en lugar del que corresponde al mensaje que acaba de llegar al receptor. Es decir, para

[1] llamada a menudo «retransmisión espuria»

confirmar el mensaje de datos con número de secuencia 0, el receptor envía una confirmación con el número de secuencia 1. En este texto tomaremos también este convenio. La confirmación necesita ese número de secuencia; si el temporizador expira antes de tiempo, al emisor puede llegarle tarde una segunda confirmación de un mensaje que ya estaba confirmado. Como indica cuál es el siguiente mensaje que espera el receptor, el emisor sabe que no se refiere al mensaje que acaba de enviar y la descarta. Sin ese número, la tomaría por la confirmación de ese mensaje aunque se hubiera perdido.

La Figura 12.1 muestra los escenarios que contempla *parada y espera*:

- El emisor envía el mensaje 0 e inicia un temporizador de retransmisión (reloj verde). El mensaje llega satisfactoriamente al receptor, que lo acepta y envía una confirmación (ack 1), que también llega correctamente. Al recibir esa confirmación, el emisor cancela el temporizador de retransmisión (reloj tachado).
- El emisor envía el mensaje 1, pero se pierde. Como consecuencia su temporizador expira (reloj rojo) y retransmite el mensaje 1 (RTX). El receptor lo confirma (ack 0) y el temporizador es cancelado.
- El segundo mensaje 0 (está reciclando el número de secuencia) llega al receptor, que lo acepta y envía una confirmación (ack 1), pero se pierde. El temporizador expira y el emisor retransmite el mensaje 0. Esta situación genera un duplicado porque el receptor ya tenía esos datos. Descarta el duplicado, pero lo confirma igualmente.

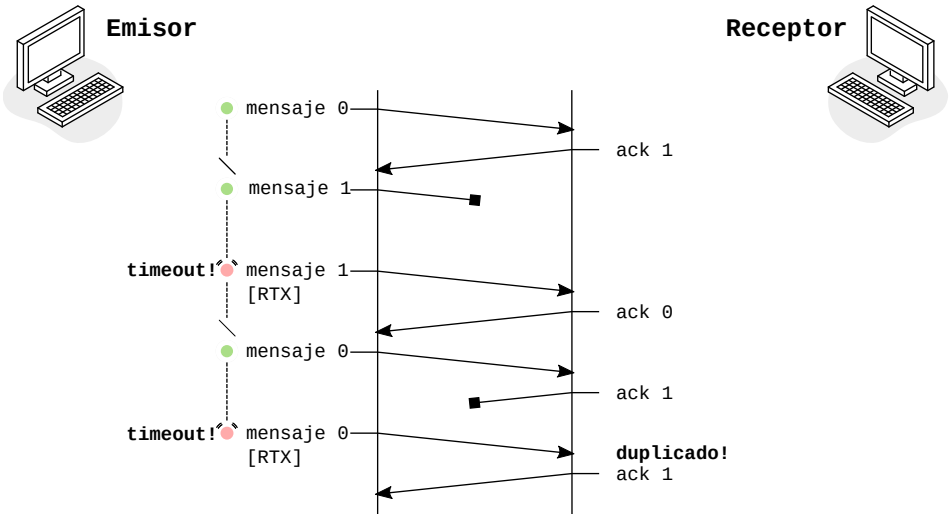


FIGURA 12.1: Diagrama de secuencia del protocolo *parada y espera*

La implementación de *parada y espera* es simple, pero ineficiente. Como el emisor debe esperar la confirmación antes de enviar el siguiente mensaje, el canal de comunicación se infrautiliza en gran medida, afectando gravemente a la velocidad de transmisión neta.

12.2. Repetición continua

Repetición continua (go back n) intenta maximizar la ocupación del canal. Para ello, el emisor envía varios mensajes consecutivos, sin esperar una confirmación antes de enviar el siguiente. Esa técnica se llama *canalización (pipelining)* y consigue una mejora sustancial del ancho de banda efectivo.

Para que funcione, el emisor tiene que guardar temporalmente el contenido de los mensajes no confirmados por si fuera necesario retransmitirlos. El espacio donde se almacenan estos mensajes suele ser una cola circular que se conoce como *buffer de envío*.

Cuando el receptor recibe un mensaje, comprueba el *checksum*, envía una confirmación al emisor y entrega el mensaje a la capa superior. Si algún mensaje se pierde, el receptor recibe mensajes fuera de secuencia. Cuando eso sucede, el receptor **descarta los mensajes** aunque sean correctos y, para cada uno de ellos, envía obligatoriamente una confirmación para el último mensaje recibido correctamente (y que respetaba la secuencia sin huecos), es decir, el emisor puede recibir múltiples confirmaciones para un mismo mensaje de datos. Se las llama «confirmaciones duplicadas».

Por tanto, las confirmaciones de repetición continua son acumulativas (§ 12.4). La confirmación con número n se aplica a todos los mensajes anteriores a n, no solo al $n-1$. No es opcional; como el receptor solo confirma el último mensaje recibido en orden, si fuera una confirmación individual bastaría con perder una para que el receptor respondiera a cada retransmisión de ese mensaje con el número del siguiente esperado. El mensaje nunca quedaría confirmado y el emisor lo retransmitiría indefinidamente.

Cada mensaje tiene su propio temporizador de retransmisión. Si expira el de un mensaje, el emisor lo retransmite junto con todos los posteriores que ya había enviado, porque el receptor los habrá descartado al llegar fuera de secuencia. Como cada transmisión vuelve a iniciar el temporizador de su mensaje, los temporizadores de esos mensajes posteriores se reinician en lugar de expirar.

Los mensajes incluyen también un número de secuencia que el receptor utiliza para confirmarlos. El espacio de números de secuencia es una potencia de 2 porque se representa en un *campo de bits* en la cabecera del mensaje.

Con k bits se obtiene un espacio de números de secuencia de 2^k mensajes, si bien no es posible enviar 2^k mensajes sin confirmación, porque provoca una situación ambigua que hay que evitar. Imagina una implementación hipotética con un número de secuencia de 2 bits (2^2 mensajes) con el siguiente escenario:

- El emisor envía los 4 mensajes posibles, con números de secuencia 0 a 3 incluidos.
- Todos los mensajes llegan correctamente a su destino.
- Todas las confirmaciones se pierden, pero el receptor lo desconoce.
- El temporizador del mensaje 0 expira.
- El emisor retransmite el mensaje 0.

En esa situación el receptor tomaría esa retransmisión por un mensaje nuevo que está reutilizando el número de secuencia 0, provocando con ello el fallo del protocolo. Para evitar esta situación, se define una *ventana de emisión* con un tamaño menor:

$$W_s \leq 2^k - 1 \tag{12.1}$$

donde:

W_s: tamaño de la ventana de emisión

La ventana de emisión determina qué mensajes se pueden enviar en cada momento, sin esperar confirmaciones y sin provocar ambigüedad. Cuando un mensaje es confirmado, su número de secuencia queda libre para ser reutilizado con un nuevo mensaje, y el buffer se reutiliza una y otra vez durante la transmisión. Esa reutilización cíclica de los números de secuencia es la *ventana deslizante (sliding window)*.

El ejemplo de la Figura 12.2 utiliza números de secuencia de $k = 2$ bits y una ventana de emisión $W_s = 3$. Empieza enviando los 3 mensajes permitidos [0–2], perdiéndose el 1. Al recibir el mensaje 0 el receptor envía ack 1; eso desliza la ventana y permite al emisor enviar el mensaje 3. Los mensajes 2 y 3 llegan al receptor fuera de secuencia (falta el 1). Por eso el receptor los descarta y envía confirmaciones ack 1.

Como el mensaje 1 no ha sido confirmado, su temporizador expira poco después y el emisor retransmite el 1 y los posteriores que ya había enviado, el 2 y el 3, cuyos temporizadores se reinician. El receptor responde con las confirmaciones 2, 3 y 0 respectivamente. La figura en el lado del emisor representa el buffer de envío. Los cuadritos verdes representan los mensajes enviados no confirmados y el marco naranja corresponde con la ventana de emisión, es decir, los mensajes que puede enviar en cada momento.

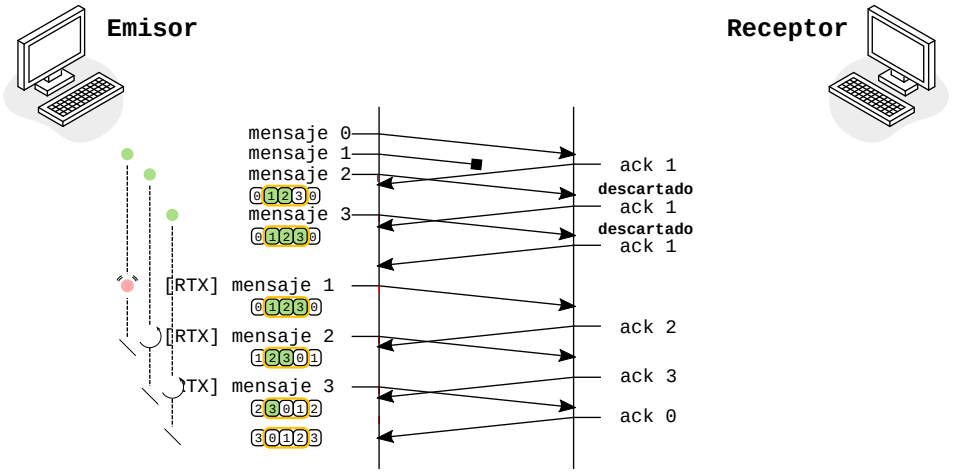


Figura 12.2: Ejemplo de *repetición continua*

Nótese en todo caso que el receptor no establece temporizadores ni retransmisiones cuando envía confirmaciones, y por supuesto el emisor no confirma las confirmaciones que recibe.

El nombre *go back n* hace referencia al hecho de que, en caso de perder un mensaje, quizá haya que «volver n mensajes atrás» y reenviarlos.

12.3. Repetición selectiva

La *repetición selectiva (selective repeat)* también aplica la técnica de canalización y tiene un temporizador de retransmisión por mensaje, pero, a diferencia de *repetición continua*, los mensajes que llegan fuera de secuencia se guardan en lugar de descartarse y cada confirmación se refiere a un único mensaje. Cuando expira un temporizador, se retransmite solo ese mensaje.

Su objetivo es evitar que el emisor tenga que retransmitir mensajes que llegaron correctamente, y de ahí el nombre del protocolo. Para lograrlo, el receptor necesita también un *buffer de recepción* que es donde guarda los mensajes fuera de secuencia. Cuando recibe el mensaje que «rellena el hueco», entrega al nivel superior todos los mensajes contiguos que haya en el buffer.

Si lo que se pierde es una confirmación, el emisor retransmite el mensaje correspondiente al expirar su temporizador, y el receptor debe volver a confirmarlo aunque ya lo tuviera.

El límite para el tamaño de la ventana es menor que en repetición continua. El receptor puede almacenar mensajes fuera de secuencia siempre que no se puedan confundir retransmisiones con mensajes de la siguiente ventana. Por eso, para un número de secuencia de k bits, el tamaño de ventana debe cumplir:

$$W_s = W_r \leq 2^{k-1} \tag{12.2}$$

donde:

> W_s es el tamaño de la ventana del emisor y W_r es el tamaño de la ventana del receptor

Un ejemplo en la Figura 12.3: con números de secuencia de $k = 3$ bits ($W_s = 2^{k-1} = 4$), un emisor envía la ventana completa (mensajes [0–3]) y se pierde el 2. El receptor confirma los mensajes 0, 1 con los ACK 1 y 2, y guarda el mensaje 3 en su buffer de recepción. Las confirmaciones recibidas por el emisor deslizan su ventana, que ahora puede enviar los mensajes 4 y 5, pero no más (tiene 4 mensajes en vuelo: [2–5]).

A diferencia de repetición continua, los mensajes fuera de secuencia no provocan confirmaciones duplicadas: el receptor guarda los mensajes 3, 4 y 5 y confirma cada uno de ellos (ACK 4, 5 y 6), de modo que el emisor cancela sus temporizadores.

Cuando expira el temporizador del mensaje 2, el emisor retransmite solo ese mensaje. Al recibirlo, el receptor entrega al nivel superior los mensajes [2–5] y lo confirma con el ACK 3.

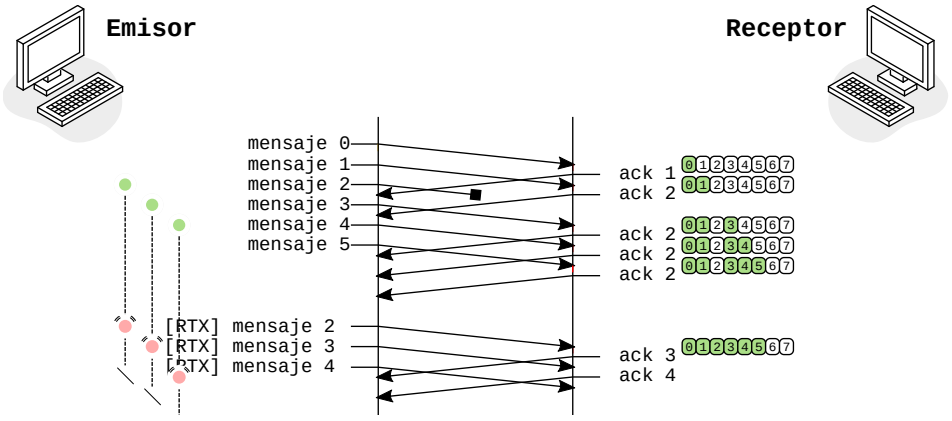


FIGURA 12.3: Ejemplo de *repetición selectiva*

12.4. Confirmación acumulativa

La *confirmación acumulativa* es un tipo diferente de confirmación. En lugar de aplicarse a un único mensaje, se aplica a todo lo anterior.

En repetición selectiva cada confirmación se refiere a un único mensaje, de modo que la pérdida de una confirmación obliga a retransmitir un mensaje que ya había llegado. Con *confirmación acumulativa*, en cambio, cada confirmación se aplica también a todos los mensajes anteriores. En repetición continua esto ocurre de forma implícita, porque el receptor solo acepta mensajes en orden (§ 12.2). En repetición selectiva, sin embargo, debe hacerse de forma explícita: como el receptor guarda mensajes fuera de secuencia, una confirmación acumulativa solo puede cubrir la parte de la secuencia que no tiene huecos.

Por ejemplo, un ACK acumulativo con número de secuencia 4 significa que todos los mensajes hasta el 3 incluido han llegado correctamente. TCP utiliza este tipo de confirmación.

La confirmación acumulativa tiene dos ventajas. Por un lado, la pérdida de una confirmación no tiene consecuencias si llega otra posterior, que la sustituye. Por otro lado, no es necesario enviar una confirmación para cada mensaje de datos. Muchos protocolos retrasan intencionadamente la confirmación (se llama «ACK retardado» o *delayed ACK*) para poder enviar una única confirmación para varios mensajes de datos.

A cambio, cuando hay un hueco en la secuencia, el receptor solo puede confirmar los mensajes anteriores al hueco. Cada mensaje que llega fuera de secuencia provoca de nuevo esa misma confirmación, por lo que el emisor recibe «confirmaciones duplicadas» y no puede saber cuáles de los mensajes posteriores han llegado.

La Figura 12.4 muestra un ejemplo con repetición selectiva y confirmación acumulativa. Con números de secuencia de $k = 3$ bits, un emisor ha enviado los mensajes [0–3] y se ha perdido el 2. El receptor envía confirmaciones con secuencia 1 y 2, que reconocen respectivamente los mensajes 0 y 1 (por ser acumulativa, bastaría con enviar la 2). Cuando llega el mensaje 3, lo almacena en su buffer de recepción y envía de nuevo una confirmación con secuencia 2. Más tarde, cuando expira el temporizador del mensaje 2, el emisor lo retransmite. El receptor, al recibirlo, dispone de la secuencia completa [0–3] y envía un ACK con secuencia 4 que confirma todos los mensajes recibidos. Mientras tanto es posible que expiren más temporizadores (el del mensaje 3 en la figura) y que haya retransmisiones innecesarias, porque el emisor no puede saber que el mensaje 3 ya llegó. Al llegar el ACK 4 se confirman todos los mensajes pendientes y se cancelan sus temporizadores

(la figura solo muestra los de los mensajes [2–3], que son los relevantes para el ejemplo).

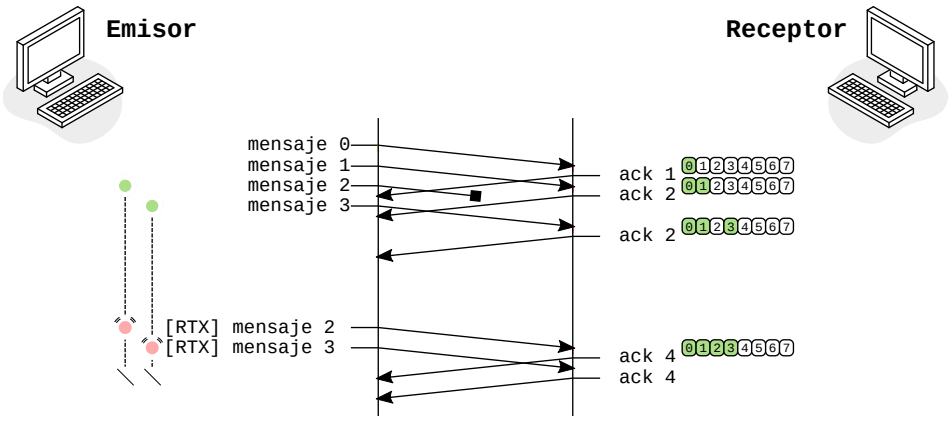


FIGURA 12.4: Ejemplo de *repetición selectiva* con confirmación acumulativa

12.5. ARQ y ancho de banda × retardo

En realidad, los tres protocolos ARQ cumplen una misma condición general:

$$W_s + W_r \leq 2^k \tag{12.3}$$

La Tabla 12.1 muestra cómo encaja esa condición en cada protocolo:

Protocolo	Restricción	Aplicación de (12.3)	W_s máx	W_r máx
Parada y espera	$W_s = W_r = 1$	$1 + 1 \leq 2^k$	1	1
Repetición continua	$W_r = 1$	$W_s + 1 \leq 2^k$	$2^k - 1$	1
Repetición selectiva	$W_s = W_r = W$	$2W \leq 2^k$	2^{k-1}	2^{k-1}

CUADRO 12.1: Aplicación de la condición $W_s + W_r \leq 2^k$ a los protocolos ARQ

Los límites de la Tabla 12.1 fijan el tamaño **máximo** de ventana que admite cada protocolo para un número de secuencia de k bits sin ambigüedad. Sin embargo, para que la canalización aproveche el canal, la ventana debe ser lo suficientemente grande como para que el emisor pueda seguir enviando mensajes continuamente, sin bloquearse a la espera de confirmaciones.

Para estimar ese tamaño intervienen dos valores:

- El *retardo de transmisión* (t_{trans}): lo que tarda el emisor en poner el mensaje completo en el canal. Depende del tamaño del mensaje (L, en bits) y de la velocidad de transmisión (R, en bits/s): $t_{\text{trans}} = L/R$.
- El *retardo de propagación* (t_{prop}): lo que tarda cada bit en recorrer el canal de extremo a extremo. Depende de la longitud del enlace (d, en metros) y de la velocidad de propagación de la señal en el medio (v, en m/s): $t_{\text{prop}} = d/v$.

Si el emisor empieza a transmitir el mensaje 0 en el instante 0, termina de enviarlo en t_{trans} y el último bit llega al receptor en $t_{\text{trans}} + t_{\text{prop}}$. Despreciando el retardo de procesamiento y el de transmisión de la confirmación (mucho más corto que el mensaje de datos), el ACK llega al emisor en $t_{\text{trans}} + 2t_{\text{prop}}$. En ese intervalo el emisor puede transmitir como mucho los W_s mensajes de su ventana, que ocupan el canal durante $W_s \cdot t_{\text{trans}}$. Para no quedar bloqueado debe cumplirse $W_s \cdot t_{\text{trans}} \geq t_{\text{trans}} + 2t_{\text{prop}}$, es decir:

$$W_s \geq 1 + 2a \tag{12.4}$$

donde:

W_s es la ventana de emisión y $a = t_{\text{prop}}/t_{\text{trans}}$

El término $2a$ indica el número de mensajes que se pueden transmitir mientras el primero llega al receptor y vuelve su ACK o, dicho de otro modo, los mensajes que se pueden enviar durante el RTT: $2 \cdot t_{\text{prop}}$. Es una simplificación; el RTT que se mide por ejemplo con `ping` incluye también los retardos de transmisión, cola y procesamiento, mientras que aquí consideramos solo propagación.

Multiplicando $2a$ por el tamaño del mensaje se obtiene el BDP (Bandwidth Delay Product), la cantidad de datos en vuelo necesaria para mantener ocupado el canal:

$$\begin{aligned} \text{BDP} &= 2a \cdot L = 2 \cdot \frac{t_{\text{prop}}}{t_{\text{trans}}} \cdot L = 2 \cdot \frac{t_{\text{prop}}}{L/R} \cdot L \\ &= R \cdot 2t_{\text{prop}} = R \cdot \text{RTT} \end{aligned} \tag{12.5}$$

El tamaño del mensaje se cancela, de modo que el BDP solo depende de la velocidad de transmisión y del retardo del enlace, y de ahí su nombre: producto ancho de banda[2] por retardo.

[2] En el sentido de velocidad de transmisión

Si la ventana es menor que $1+2a$ mensajes, el emisor transmite su ventana completa y debe esperar la primera confirmación. En ausencia de errores, el aprovechamiento del canal (U), o fracción de tiempo que el emisor transmite, es:

$$U = \begin{cases} 1 & \text{si } W_s \geq 1+2a \\ \dfrac{W_s}{1+2a} & \text{si } W_s < 1+2a \end{cases} \tag{12.6}$$

Parada y espera es $W_s = 1$, con $U = 1/(1+2a)$, que se corresponde con su bajo rendimiento en canales con mucho retardo o con una velocidad de transmisión alta.

Por tanto, la ventana tiene dos límites: $W_s \geq 1+2a$ es el mínimo para que el protocolo sea eficiente y $W_s \leq W_{s,\text{máx}}$ es el máximo para que sea correcto y que depende del protocolo según la Tabla 12.1 ($2^k - 1$ para repetición continua y 2^{k-1} para repetición selectiva):

$$1+2a \leq W_s \leq W_{s,\text{máx}} \tag{12.7}$$

Un ejemplo: en un enlace de fibra óptica[3] de 1000 km a 10 Mbps con mensajes de 1000 bytes:

- $t_{\text{trans}} = L/R = 8000$ bits/10 Mbps $= 0{,}8$ ms
- $t_{\text{prop}} = d/v \approx 10^6$ m/$(2 \cdot 10^8$ m/s$) = 5$ ms
- $a = 5/0{,}8 = 6{,}25$
- $1+2a = 13{,}5$. En la ventana deben caber al menos 14 mensajes.

La Tabla 12.2 recoge el valor mínimo de k que necesita cada protocolo en este enlace y en un enlace satelital ($t_{\text{prop}} \approx 250$ ms) para conseguir un aprovechamiento óptimo.

Es posible utilizar ventanas mayores de $1+2a$, pero no mejora el aprovechamiento del canal y además aumenta el coste: buffers de envío mayores (y de recepción, en repetición selectiva), un campo de número de secuencia más largo en la cabecera y, en repetición continua, más mensajes que retransmitir tras cada pérdida.

[3] La luz se propaga en la fibra a unos $2 \cdot 10^8$ m/s.

	Fibra (1000 km)	Satélite
Retardo de propagación (t_{prop})	5 ms	250 ms
$a = t_{\text{prop}}/t_{\text{trans}}$ (mensajes)	6,25	312,5
$W_{s,\text{mín}} = 1 + 2a$	13,5	626
k en repetición continua	4	10
$W_{s,\text{máx}} = 2^k - 1$	15	1023
k en repetición selectiva	5	11
$W_{s,\text{máx}} = 2^{k-1}$	16	1024

CUADRO 12.2: Elección de k para un enlace de fibra y otro por satélite

12.6. Confirmación negativa

Es posible mejorar la repetición selectiva, aunque, como siempre, a cambio de complejidad adicional. El receptor puede utilizar un nuevo tipo de confirmación —la confirmación negativa o NACK— para notificar explícitamente cuál es el mensaje corrupto o perdido. De ese modo, el emisor puede retransmitir inmediatamente ese mensaje en lugar de esperar a que expire su temporizador, aprovechando mejor el canal.

Retomando el ejemplo de la Figura 12.4 —números de secuencia de 3 bits, mensajes [0–3] enviados con pérdida del mensaje 2—, con NACK el receptor no necesita esperar a que el hueco se rellene para reaccionar: en cuanto recibe el mensaje 3 fuera de secuencia detecta el hueco en 2 y envía un NACK con ese número, junto con la confirmación que le corresponda. El emisor retransmite el mensaje 2 de inmediato en lugar de esperar a que expire su temporizador, que se cancela tan pronto como llega el NACK. El resto de la recuperación es igual que antes: al completar la secuencia [0–3], el receptor envía un ACK con secuencia 4 confirmando todos los mensajes pendientes.

12.7. Confiabilidad en comunicaciones dúplex

Los protocolos de confiabilidad tratan la comunicación solo en un sentido de la comunicación. Hemos hablado todo el tiempo de «emisor» y «receptor». En realidad se explica de este modo por simplicidad y para ayudar a entender su funcionamiento. Nada impide que en un canal dúplex ambos extremos actúen como emisores y receptores a la vez. En ese caso, esos protocolos trabajan de forma independiente para cada uno de los sentidos

de la comunicación empleando ventanas y números de secuencia independientes.

En realidad, utilizar protocolos de confiabilidad en ambos sentidos permite optimizar algunas situaciones. Por ejemplo, resulta ineficiente enviar varios mensajes pequeños consecutivos. Por eso, cuando uno de los extremos necesita enviar una confirmación, que solo ocupa una simple cabecera, puede esperar «un poco» y aprovechar un mensaje de datos saliente para colocar la confirmación en su cabecera. A este tipo de optimizaciones se las llama *piggybacking* y es tan habitual que los formatos de las cabeceras se diseñan específicamente para hacerlo posible.

Y ¿qué más?

Los protocolos ARQ son la base de la mayoría de los protocolos que proporcionan confiabilidad en distintas capas. Algunos de estos protocolos son IEEE 802.11 (WiFi), HDLC, Bluetooth o LTE, que son protocolos de la capa de enlace[4], TCP o SCTP que son protocolos de transporte, o TFTP que es un protocolo de aplicación. Sin duda alguna, el más importante de todos estos protocolos es TCP. Por eso trataremos en detalle su funcionamiento y características en el capítulo 13.

[4]En realidad, algunos de estos protocolos abarcan varias capas, pero los mecanismos de confiabilidad los ofrecen en la capa de enlace.